AI Film Pipeline — Complete Guide

From Renting an Instance to Final MP4

Pulse & Paradox | Documentary & Storytelling Series

Pipeline Overview

This guide covers the complete workflow to generate an AI film from scratch on a fresh Vast.ai instance. Follow every step in order. Do not skip steps.

Step	What Happens	Time
1	Rent Vast.ai instance	5 min
2	Install all libraries	10 min
3	Download Juggernaut XL v9 model	10 min
4	Save story script as JSON	2 min
5	Generate audio with F5-TTS	20 min
6	Generate images with Juggernaut XL	20 min
7	Generate video clips with FFmpeg	45 min
8	Check all clips	2 min
9	Generate background music	2 min
10	Generate subtitles	2 min
11	Final assembly	10 min

STEP 1 — Rent a Vast.ai Instance

Go to vast.ai and rent an instance with these minimum specs:

- GPU: RTX 3090 / 4090 / 5060 Ti or better
- VRAM: 16GB minimum
- Disk: 100GB minimum
- Image: vastai/comfy_v0.15.0-cuda-12.9-py312 (or any PyTorch image)
- Estimated cost: ~$0.10/hr

> IMPORTANT: Vast.ai instances lose all data when deleted. Download your final video before deleting. Do not leave instance running idle — it charges per hour.

STEP 2 — Install All Libraries

Run this exact command in the terminal. Order matters — numpy must install first:

```
pip install numpy --break-system-packages && \ pip install diffusers transformers accelerate safetensors \ huggingface_hub soundfile scipy --break-system-packages && \ pip install f5-tts --break-system-packages && \ echo "ALL LIBS DONE!"
```

Libraries installed and what they do:

Library	Purpose
numpy	Required by everything — install first
diffusers	Run Juggernaut XL image generation
transformers + accelerate + safetensors	Required by diffusers
huggingface_hub	Download models from HuggingFace
soundfile + scipy	Save and process audio files
f5-tts	Voice/narration generation

STEP 3 — Download Juggernaut XL v9

No HuggingFace login needed. Run:

```
mkdir -p /workspace/models/juggernaut && \ cat > /tmp/dl_juggernaut.py << 'EOF' from huggingface_hub import hf_hub_download hf_hub_download( repo_id="RunDiffusion/Juggernaut-XL-v9", filename="Juggernaut-XL_v9_RunDiffusionPhoto_v2.safetensors", local_dir="/workspace/models/juggernaut" ) print("Juggernaut Done!") EOF python3 /tmp/dl_juggernaut.py
```

Model size: ~7GB. Downloads in ~10 minutes. Saved to /workspace/models/juggernaut/

WHY Juggernaut XL v9: Best cinematic quality without requiring HuggingFace login. FLUX.1-dev requires account verification and often fails on Vast.ai network restrictions.

STEP 4 — Save Your Story Script

Each scene needs three fields:

- narration — what the voice will say
- image_prompt — detailed cinematic description for Juggernaut XL
- scene_type — always 'nonhuman' (for wide angle shots without faces)

Image prompt formula for best results:

```
"wide angle [shot type] of [subject], [lighting], [atmosphere], cinematic documentary 4k"
```

Duration guide: ~14 characters per second of narration. For 20 minutes target 16,800+ characters total across all scenes.

Save script with:

```
json.dump(scenes, open('/workspace/film_script.json', 'w'))
```

STEP 5 — Generate Audio with F5-TTS

F5-TTS requires a reference audio file to clone a voice. We use the built-in reference that ships with f5-tts:

Reference audio path:

```
/venv/main/lib/python3.12/site-packages/f5_tts/infer/examples/basic/basic_ref_en.wav
```

Reference text: Some call me nature, others call me mother nature.

Generation script:

```
cat > /tmp/gen_audio.py << 'PYEOF' import json, os, soundfile as sf from f5_tts.api import
F5TTS scenes = json.load(open('/workspace/film_script.json'))
os.makedirs('/workspace/film_output/audio', exist_ok=True) tts = F5TTS() ref_audio =
'/venv/main/lib/python3.12/site-packages/f5_tts/infer/examples/basic/basic_ref_en.wav'
ref_text = "Some call me nature, others call me mother nature." for i, scene in
enumerate(scenes): out = f'/workspace/film_output/audio/scene_{i:03d}.wav' if
os.path.exists(out): print(f" skip {i+1}"); continue print(f" {i+1}/{len(scenes)}:
{scene['narration'][:55]}...") tts.infer(ref_file=ref_audio, ref_text=ref_text,
gen_text=scene['narration'], file_wave=out) print(f" done") print("All audio done!")
PYEOF python3 /tmp/gen_audio.py
```

After generation, verify duration:

```
python3 -c " import os, subprocess total =
sum(float(subprocess.run(['ffprobe','-v','error','-show_entries',
'format=duration','-of','default=noprint_wrappers=1:nokey=1',
f'/workspace/film_output/audio/scene_{i:03d}.wav'],
capture_output=True,text=True).stdout.strip()) for i in range(NUM_SCENES) if
os.path.exists(f'/workspace/film_output/audio/scene_{i:03d}.wav')) print(f'Total:
{total:.1f}s ({total/60:.1f} min)') "
```

Replace NUM_SCENES with your actual scene count.

STEP 6 — Generate Images with Juggernaut XL

Resolution: 1280x720 (HD). Steps: 35. Guidance: 7.5

```
cat > /tmp/gen_images.py << 'PYEOF' import json, os, torch from diffusers import
StableDiffusionXLPipeline scenes = json.load(open('/workspace/film_script.json'))
os.makedirs('/workspace/film_output/images', exist_ok=True) pipe =
StableDiffusionXLPipeline.from_single_file(
'/workspace/models/juggernaut/Juggernaut-XL_v9_RunDiffusionPhoto_v2.safetensors',
torch_dtype=torch.float16 ).to('cuda') pipe.enable_vae_tiling()
pipe.enable_attention_slicing() for i, scene in enumerate(scenes): out =
f'/workspace/film_output/images/scene_{i:03d}.png' if os.path.exists(out): print(f" skip
{i+1}"); continue print(f" {i+1}/{len(scenes)}: generating...") pipe(
scene['image_prompt'], negative_prompt="blurry, watermark, text, cartoon, anime, ugly,
low quality, nsfw, people, faces", width=1280, height=720, num_inference_steps=35,
guidance_scale=7.5 ).images[0].save(out) print(f" saved") print("All images done!") PYEOF
python3 /tmp/gen_images.py
```

Negative prompt always includes: blurry, watermark, text, cartoon, anime, ugly, low quality, nsfw, people, faces

STEP 7 — Generate Video Clips with FFmpeg

Uses zoompan filter for cinematic camera motion. 12 different movements rotate across scenes: push in, pull out, pan left/right, tilt up/down, diagonal drifts. Proven settings: zoom+0.0008, max zoom 1.3, 4x scale, 25fps.

```
cat > /tmp/gen_clips.py << 'PYEOF' import json, os, subprocess scenes =
json.load(open('/workspace/film_script.json')) os.system('rm -f
/workspace/film_output/clip_*.mp4') def get_motion(i, dur): tf = int(dur * 25) motions =
[ f"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='iw/2-(iw/zoom/2)':y='ih/2-(ih/zoom/
2)':fps=25:d={tf}:s=1280x720", f"scale=-2:4*ih,zoompan=z='if(lte(zoom,1.0),1.3,max(1.0,zo
om-0.0008))':x='iw/2-(iw/zoom/2)':y='ih/2-(ih/zoom/2)':fps=25:d={tf}:s=1280x720", f"scale
=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='0':y='ih/2-(ih/zoom/2)':fps=25:d={tf}:s=1280
x720", f"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='iw-iw/zoom':y='ih/2-(ih/zoom/2
)':fps=25:d={tf}:s=1280x720", f"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='iw/2-(i
```

```
w/zoom/2)':y='0':fps=25:d={tf}:s=1280x720", f"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.
3)':x='iw/2-(iw/zoom/2)':y='ih-ih/zoom':fps=25:d={tf}:s=1280x720",
f"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='0':y='0':fps=25:d={tf}:s=1280x720", f
"scale=-2:4*ih,zoompan=z='min(zoom+0.0008,1.3)':x='iw-iw/zoom':y='ih-ih/zoom':fps=25:d={t
f}:s=1280x720", f"scale=-2:4*ih,zoompan=z='if(lte(zoom,1.0),1.3,max(1.0,zoom-0.0008))':x=
'0':y='ih/2-(ih/zoom/2)':fps=25:d={tf}:s=1280x720", f"scale=-2:4*ih,zoompan=z='if(lte(zoo
m,1.0),1.3,max(1.0,zoom-0.0008))':x='iw-iw/zoom':y='ih/2-(ih/zoom/2)':fps=25:d={tf}:s=128
0x720", f"scale=-2:4*ih,zoompan=z='if(lte(zoom,1.0),1.3,max(1.0,zoom-0.0008))':x='iw/2-(i
w/zoom/2)':y='0':fps=25:d={tf}:s=1280x720", f"scale=-2:4*ih,zoompan=z='if(lte(zoom,1.0),1
.3,max(1.0,zoom-0.0008))':x='0':y='0':fps=25:d={tf}:s=1280x720", ] return motions[i %
len(motions)] for i, scene in enumerate(scenes): audio =
f'/workspace/film_output/audio/scene_{i:03d}.wav' img =
f'/workspace/film_output/images/scene_{i:03d}.png' clip =
f'/workspace/film_output/clip_{i:03d}.mp4' if not os.path.exists(audio) or not
os.path.exists(img): print(f" missing files for clip {i}"); continue dur =
float(subprocess.run(['ffprobe','-v','error','-show_entries','format=duration',
'-of','default=noprint_wrappers=1:nokey=1',audio],
capture_output=True,text=True).stdout.strip()) + 0.3 vf =
f"{get_motion(i,dur)},fade=t=in:st=0:d=0.5,fade=t=out:st={dur-0.5}:d=0.5" r =
subprocess.run(['ffmpeg','-y','-loop','1','-t',str(dur),'-i',img,
'-i',audio,'-vf',vf,'-map','0:v','-map','1:a',
'-c:v','libx264','-crf','18','-c:a','aac','-b:a','192k',
'-pix_fmt','yuv420p','-r','25','-t',str(dur),clip], capture_output=True,text=True) size =
os.path.getsize(clip) if os.path.exists(clip) else 0 print(f" {'OK' if size>50000 else
'FAILED'} clip {i+1}/{len(scenes)} ({size//1024}KB)") print("All clips done!") PYEOF
python3 /tmp/gen_clips.py
```

STEP 8 — Check All Clips

Replace NUM_SCENES with your scene count:

```
python3 << 'EOF' import os NUM_SCENES = 50 broken = [i for i in range(NUM_SCENES) if not
os.path.exists(f'/workspace/film_output/clip_{i:03d}.mp4') or
os.path.getsize(f'/workspace/film_output/clip_{i:03d}.mp4') < 50000] print(f"Broken:
{broken}" if broken else "ALL CLIPS OK!") EOF
```

STEP 9 — Generate Background Music

Generates locally using numpy — no internet needed. Cinematic tension curve: quiet intro, builds through middle, peaks at climax, resolves at end. Mixed at 10% volume under narration.

```
cat > /tmp/gen_music.py << 'PYEOF' import numpy as np from scipy.io import wavfile sr =
44100; duration = 1200 t = np.linspace(0, duration, sr * duration) envelope =
np.zeros(duration) for sec in range(duration): p = sec / duration if p < 0.1:
envelope[sec] = 0.2 + p * 3 elif p < 0.4: envelope[sec] = 0.5 + p * 0.8 elif p < 0.65:
envelope[sec] = 0.82 elif p < 0.75: envelope[sec] = 0.82 + (p-0.65)*2 else: envelope[sec]
= 1.0 - (p-0.75)*2.5 env = np.interp(np.linspace(0,duration,sr*duration),
np.arange(duration), envelope) bass = 0.28 * np.sin(2*np.pi*48*t) mid = 0.14 *
np.sin(2*np.pi*96*t + np.sin(2*np.pi*0.08*t)) high = 0.07 * np.sin(2*np.pi*192*t +
np.sin(2*np.pi*0.04*t)) pulse = 0.5 + 0.5*np.sin(2*np.pi*0.06*t) tens = 0.10 *
np.sin(2*np.pi*144*t) * pulse string = 0.06 * np.sin(2*np.pi*220*t +
np.sin(2*np.pi*0.12*t)) * pulse music = (bass+mid+high+tens+string)*env music =
(music/np.max(np.abs(music))*0.55*32767).astype(np.int16)
wavfile.write('/workspace/film_output/music.wav', sr, music) print("Music done!") PYEOF
python3 /tmp/gen_music.py
```

STEP 10 — Generate Subtitles

Auto-generated from narration text. 8 words per subtitle line. Timed to match audio duration per scene.

```
cat > /tmp/gen_subtitles.py << 'PYEOF' import json, subprocess, re, os scenes =
json.load(open('/workspace/film_script.json')) def fmt(s):
h,m=int(s//3600),int((s%3600)//60) return
f"{h:02d}:{m:02d}:{int(s%60):02d},{int((s%1)*1000):03d}" srt_lines = []; current_time =
0.0 for i, scene in enumerate(scenes): audio =
f'/workspace/film_output/audio/scene_{i:03d}.wav' dur =
float(subprocess.run(['ffprobe','-v','error','-show_entries','format=duration',
'-of','default=noprint_wrappers=1:nokey=1',audio],
capture_output=True,text=True).stdout.strip()) words =
re.sub(r'\[.*?\]','',scene['narration']).strip().split() chunks = [' '.join(words[j:j+8])
for j in range(0,len(words),8)] chunk_dur = dur / max(len(chunks),1) for k,chunk in
enumerate(chunks): start = current_time + k*chunk_dur
srt_lines.append(f"{len(srt_lines)+1} {fmt(start)} --> {fmt(start+chunk_dur)} {chunk} ")
current_time += dur + 0.3 with open('/workspace/film_output/subtitles.srt','w') as f:
f.write(' '.join(srt_lines)) print(f"Subtitles done! {len(srt_lines)} entries") PYEOF
python3 /tmp/gen_subtitles.py
```

STEP 11 — Final Assembly

Combines all clips + music + subtitles + color grade into one MP4. Color grade: dark cinematic teal/blue documentary look (contrast +1.1, saturation 0.75, blue channel +12%).

Replace NUM_SCENES and OUTPUT_NAME before running:

```
cat > /tmp/final_assemble.py << 'PYEOF' import subprocess, os NUM_SCENES = 50 OUTPUT_NAME
= "EPISODE_FINAL" clips = [f"file '/workspace/film_output/clip_{i:03d}.mp4'" for i in
range(NUM_SCENES) if os.path.exists(f'/workspace/film_output/clip_{i:03d}.mp4')]
print(f"Found {len(clips)} clips") with open('/workspace/film_output/concat.txt','w') as
f: f.write(' '.join(clips)) print("Merging clips...")
subprocess.run(['ffmpeg','-y','-f','concat','-safe','0',
'-i','/workspace/film_output/concat.txt','-c','copy',
'/workspace/film_output/film_raw.mp4'], capture_output=True) total_dur =
float(subprocess.run(['ffprobe','-v','error','-show_entries','format=duration',
'-of','default=noprint_wrappers=1:nokey=1','/workspace/film_output/film_raw.mp4'],
capture_output=True,text=True).stdout.strip()) print(f"Duration: {total_dur:.1f}s
({total_dur/60:.1f} min)") subs = '/workspace/film_output/subtitles.srt' music =
'/workspace/film_output/music.wav' style = 'FontSize=20,PrimaryColour=&Hffffff,OutlineCol
our=&H000000,Outline=2,Shadow=1,Alignment=2,Bold=1' final =
f'/workspace/film_output/{OUTPUT_NAME}.mp4' print("Adding music + subtitles + color
grade...") r = subprocess.run([ 'ffmpeg','-y',
'-i','/workspace/film_output/film_raw.mp4', '-stream_loop','-1','-i',music,
'-filter_complex', f"[0:v]eq=contrast=1.1:brightness=-0.05:saturation=0.75,"
f"colorchannelmixer=rr=0.88:gg=0.94:bb=1.12,"
f"subtitles='{subs}':force_style='{style}'[v];"
"[1:a]volume=0.10[music];[0:a][music]amix=inputs=2:duration=first[a]",
'-map','[v]','-map','[a]', '-c:v','libx264','-crf','18','-c:a','aac','-b:a','192k',
'-t',str(total_dur), final ], capture_output=True, text=True) if os.path.exists(final):
print(f"DONE! {os.path.getsize(final)//1024//1024}MB | {final}") else: print(f"ERROR:
{r.stderr[-300:]}") PYEOF python3 /tmp/final_assemble.py
```

Quick Reference — Common Issues

Problem	Fix
Clips are 0KB / too small	Re-run gen_clips.py — zoompan failed silently
Audio duration mismatch	Check ffprobe on individual audio file
CUDA out of memory	Add pipe.enable_attention_slicing() and pipe.enable_vae_tiling()
FLUX model won't download	Use Juggernaut XL instead — no login needed
F5-TTS needs reference audio	Use built-in: /venv/.../f5_tts/infer/examples/basic/basic_ref_en.wav
Final video too short	Expand script — need ~14 chars/sec for target duration
Subtitles not showing	Check subs path has no spaces, use absolute path in FFmpeg

Script Duration Guide

Target Duration	Scenes Needed	Total Characters
6-7 minutes	~25 scenes	~5,000 chars
10-12 minutes	~40 scenes	~8,400 chars
14-15 minutes	~50 scenes	~11,800 chars
20 minutes	~70 scenes	~16,800 chars